ЛАБОРАТОРНА РОБОТА №5
Операційний рівень системних вимог: специфікація прецедентів (Use Cases) та аналіз інженерних станів

Мета роботи: На основі спроєктованого гнучкого Agile-беклогу (з ЛР №4) здійснити перехід до системної архітектури продукту; навчитися проводити аналітичне каскадування та агрегацію користувацьких історій (User Stories) у монолітні Enterprise-прецеденти (Use Cases); опанувати логіку проєктування інженерних станів системи за принципом контракту (Preconditions / Postconditions) та алгоритмізацію розгалужень процесів (Alternative Flows); переорієнтувати генеративний ШІ на роль суворого зовнішнього кібераудитора для проведення краш-тесту логіки та навчитися описувати відмовостійкі сценарії обробки критичних технічних аварій (Exception Flows) із захистом цілісності даних у базі.

Тривалість: 2 аудиторні пари згідно з розкладом.

ФОРМА ЗДАЧІ ТА ЗАХИСТУ РОБОТИ (DEMO DAY)

Захист результатів відбувається безпосередньо під час занять у форматі командної експрес-презентації (Demo Day) із суворим часовим регламентом, що імітує внутрішні технічні рев'ю в IT-компаніях.

- Регламент Блоку 1 (Завдання 1–2) – 4 хвилин:
  - Що перевіряється: Якість аналітичного мислення при групуванні вимог. Наявність наскрізного трасування між Jira та Confluence (активні Jira Macro лінки). Оцінюється покроковий алгоритм Happy Path та залізобетонність інженерних контрактів (Preconditions / Postconditions) без «гуманітарних» описів емоцій чи інтерфейсу.
  - Рольовий ценз: Студент А (BA) аргументує логіку об'єднання історій, межі прецеденту та дії первинних акторів. Студент Б (PM) захищає технічні реакції системи, взаємодію з другорядними API/шлюзами та незворотні зміни прапорців/статусів у базі даних.
- Регламент Блоку 2 (Завдання 3–4) – 4 хвилини:
  - Що перевіряється: Готова логічна матриця бізнес-розгалужень (Alternative Flows) 1-го та 2-го типів, де чітко відкинуто «інтерфейсне сміття» фронтенду. Також перевіряється якість проведеного ШІ-краш-тесту на сторінці Notion (журнал логів суперечки) та спроєктовані в Confluence сценарії порятунку даних у разі збоїв (Exception Flows).
  - Рольовий ценз: Студент А (BA) захищає користувацькі альтернативи та логіку відображення заспокійливих і зрозумілих вікон помилок на UI. Студент Б (PM) аргументує логіку «під капотом»: системне скасування операцій, автоматичні повтори запитів та фіксацію архітектурних рішень у Decision Log.

ПОРЯДОК ВИКОНАННЯ

ЗАВДАННЯ №1. Системне каскадування: Агрегація User Stories у монолітні Use Cases

Мета: Навчитися проводити ревізію Agile-беклогу, виявляти системні та архітектурні зв'язки між окремими вимогами користувачів і здійснювати їх каскадування (агрегацію) у високорівневі Enterprise-прецеденти (Use Cases), чітко визначаючи межі проєктованої системи.

Базовий інструментарій: Jira, Confluence.

Покроковий алгоритм виконання:

1. Ревізія беклогу та аналітичне групування User Story.

- Студент А (BA): Переглядає весь пул User Stories (розроблених у ЛР№4), абстрагується від дрібних інтерфейсних дій, та групує декілька взаємопов’язаних Stories у наскрізні системні процеси навколо однієї великої бізнес-функції системи (прецедент, Use Case). Аргументує, чому саме ці Stories формують обраний прецедент з погляду цінності для бізнесу та кінцевого користувача.

  Приклад групування Stories: «Я як клієнт хочу ввести промокод», «Я як клієнт хочу обрати оплату карткою» та «Я як клієнт хочу підтвердити замовлення» мають бути агреговані в один монолітний Use Case: «Оформлення та оплата замовлення».

2. Визначення меж та ідентифікація Акторів.

- Студент А (BA): Для кожного сформованого Use Case визначає Primary Actor (суб'єкт (людина або автономна система), який безпосередньо ініціює прецедент для досягнення своєї мети) та Secondary Actors (зовнішні сервіси, платіжні шлюзи, апаратні модулі або сторонні API, з якими система повинна взаємодіяти під час виконання цього прецеденту). Для визначених Primary Actor, описує їхні кінцеві бізнес-цілі в межах даного Use Case.

3. Налаштування наскрізного трасування.

- Студент Б (PM): Створює у Confluence в документі BRD новий Блок 4 «Системні прецеденти» та розгортає Розділ 4.1. «Специфікації системних прецедентів (Use Cases)» у вигляді Markdown-таблиці. За допомогою Jira Macro інструментів безшовно пов'язує Use Cases із User Stories в Jira.

Приклад таблиці в Confluence:

ID Use Case	Назва прецеденту (Enterprise стандарт)	Первинний Актор	Вторинні Актори (Зовнішні API / Системи)	Пов'язані User Stories (Jira Macro links)	Обґрунтування меж прецеденту (Чому ці Stories об'єднані)
UC-01	Оформлення замовлення та ініціація оплати	Клієнт	LiqPay API, Poster POS API, Firebase Cloud Messaging	[PROJ-12] [PROJ-13]	Обидві історії виконуються в межах однієї неподільної сесії користувача та завершуються єдиним фінансовим результатом для БД.

ЗАВДАННЯ №2. Проєктування транзакційного еталону та формалізація інженерних станів (Happy Path, Preconditions & Postconditions)

Мета: Навчитися мислити категоріями станів системи (State Machine) та транзакційності даних; спроєктувати покроковий еталонний алгоритм взаємодії (Main Success Scenario / Happy Path) з чітким описом технічних реакцій сервера та формалізувати жорсткі інженерні контракти – передумови (Preconditions) та постумови (Postconditions) виконання прецеденту.

Базовий інструментарій (Tech Stack): Confluence Cloud (документ BRD, новий Розділ 4.2 «Специфікація еталону та інженерних станів прецедентів»).

Покроковий алгоритм виконання:

1. Ініціація Use Case Specification.

- Студент А (BA): Для сформованих Use Cases у Confluence (Розділ 4.2) розробляється таблична структура за стандартом (шаблон Use Case Specification).
  - ✓ ID та Назва прецеденту: (наприклад, UC-01: Оформлення замовлення та синхронізація з кухнею ресторану)
  - ✓ Первинний Актор (Primary Actor): (хто ініціює)
  - ✓ Вторинні Актори (Secondary Actors): (зовнішні системи/API)
  - ✓ Короткий опис: (яку бізнес-ціль замовника вирішує цей сценарій)
  - ✓ Preconditions (Передумови): повний перелік вимог до стану системи та даних до початку виконання сценарію.
  - ✓ Таблиця Main Success Scenario (Happy Path):

Крок	Дія Актора (Заповнює BA)	Реакція та відповідь системи (Заповнює PM)
1.		
…		

  - ✓ Postconditions (Постумови): повний перелік гарантованих системних та інфраструктурних змін після успішного завершення сценарію.

2. Формалізація інженерних передумов (Preconditions).

Команда фіксує за яких чітких умов система та база даних готові до старту взаємодії користувачів за визначеними Use Cases.

- Студент А (BA): Фокусується на продуктових та користувацьких станах. Він прописує, в якому статусі має перебувати сесія користувача (наприклад, «Користувач авторизований, роль – Клієнт»), які бізнес-об'єкти мають бути сформовані (наприклад, «У кошику є мінімум 1 доступна страва, сума замовлення > 0»), та які користувацькі інтерфейси мають бути відкриті для старту сценарію взаємодії (прецеденту).
- Студент Б (PM): Фокусується на технічних прапорцях, статусах системи та умовах транзакційності в БД. Він формалізує передумови з погляду інфраструктури (наприклад, «Зв'язок із сервером Poster POS активний, стоп-листи оновлені», «Сервер LiqPay повернув статус 200 на первинний пінг»).

3. Алгоритмізація Happy Path (Еталону).

Для визначених Use Cases команда розписує Main Success Scenario: покроковий ланцюжок ідеального виконання бізнес-мети системи, де кожен крок є логічним наслідком попереднього (без збоїв, помилок та відхилень). Кожен крок описується за схемою: «Крок: Дія Актора → Реакція та відповідь системи».

- Студент А (BA): Відповідає за бізнес-контур та користувацький досвід, розписує ліву частину таблиці сценарію: Дії Актора (Actor Actions). Що саме людина обирає, вводить або який тригер ініціює в інтерфейсі (без деталізації дизайну кнопок, суто логічний намір користувача).
- Студент Б (PM): Відповідає за технічний та інфраструктурний контур, розписує праву частину таблиці сценарію: Реакція та відповідь системи (System Responses). Вказує логіку сервера: які API-методи тригеряться, які SQL/NoSQL запити надсилаються до БД, які внутрішні перевірки чи обчислення виконує backend перед тим, як повернути відповідь на фронтенд.

4. Формалізація інженерних постумов (Postconditions).

Команда визначає повний перелік гарантованих системних та інфраструктурних змін після успішного завершення Use Cases.

- Студент Б (PM): Описує, які незворотні сліди в системі залишили Use Cases після того, як успішно завершився останній крок їх Happy Path.

ПРИМІТКА: Щоб правильно визначити постумови, студенти мають перевірити свій Use Case за трьома інженерними фільтрами:

1. Зміна станів у Базі Даних: Що записалося в БД системи? Якщо Use Case успішний, дані не могли залишитися незмінними.
   - Погано: «Замовлення оформлено».
   - Просто і правильно: Статус замовлення в таблиці orders змінився з Created на Paid. Створився новий рядок у таблиці financial_logs.
2. Контракти із зовнішніми системами та API: Оскільки наш додаток не живе у вакуумі (це відкрита система), кого ми розбудили своїм прецедентом зовні? Кому ми передали естафету?
   - Погано: «Кухня ресторану дізналася про замовлення».
   - Просто і правильно: Наш сервер успішно відправив POST-запит (JSON-пакет) на API ресторану-партнера. Гроші списалися через API платіжного шлюзу.
3. Автоматичні тригери та сповіщення: Які автоматичні процеси запустила система без участі людини?
   - Погано: «Кур'єр пішов забирати їжу».
   - Просто і правильно: Система згенерувала і надіслала push-сповіщення через Firebase у додаток кур'єра про наявність нового замовлення.

ЗАВДАННЯ №3. Алгоритмізація альтернативних сценаріїв та логіка серверних перевірок

Мета: Навчитися проєктувати логічні розгалуження в системі (Alternative Flows) за принципом комп'ютерної логіки if/else; чітко відрізняти прості помилки введення в інтерфейсі (які виправляє фронтенд) від суворих обмежень бізнесу (які перевіряє база даних).

Базовий інструментарій: Confluence (документ BRD, Розділ 4.3 «Альтернативні сценарії прецеденту [ID]»).

Покроковий алгоритм виконання:

1. Визначення точок розгалуження та логічних помилок.

Команда аналізує Happy Path сформованих Use Cases із Завд.2 та визначає кроки, де процес може піти іншим шляхом або де користувач може порушити правила системи. Кожне відгалуження має чітко посилатися на конкретний крок основного Happy Path, з якого воно стартує, та вказувати крок, на який система повертає користувача після успішного відпрацювання альтернативи.

Для кожного Use Cases необхідно передбачити не менше 2-х альтернативних потоків Типу 1, та не менше 2-х – типу 2.

- ✓ Тип 1. Зміна бізнес-параметрів (Користувацький або доменний вибір): Ситуація, де користувач свідомо обирає інший шлях реалізації мети (наприклад: зміна оплати карткою на готівку, перемикання з доставки кур'єром на самовивіз із ресторану тощо).
- ✓ Тип 2. Помилка бізнес-валідації даних (Користувацька логічна помилка): Ситуація, коли користувач ввів дані, які з погляду інтерфейсу (UI) є коректними (наприклад, текст або цифри), але серверна логіка та база даних їх відхиляють (наприклад: введення промокоду, який уже закінчився чи був активований; спроба оформити доставку на адресу, яка за геозонуванням перебуває поза межами обслуговування ресторану; замовлення страви, яка щойно потрапила в стоп-лист у Poster POS тощо).

Примітка: Прості інтерфейсні помилки фронтенду (не заповнено обов'язкове поле, введено літери замість цифр) сюди не пишуться, оскільки вони блокуються на рівні UI та є типовими для системи.

2. Фіксація результатів. У Confluence (Розділ 4.3) розробляється таблична структура за шаблоном.

- Студент А (BA) розписує ліву колонку таблиці (Дії актора в інтерфейсі та тексти помилок).
- Студент Б (PM) розписує праву колонку таблиці (яка таблиця в БД перевіряється та які статуси змінюються).

Специфікація альтернативного потоку (Alternative Flow Specification):

- ✓ ID потоку: (наприклад, AF-01A)
- ✓ Старт із кроку Happy Path: (вказати номер кроку, з якого повернули, напр. Крок №3)
- ✓ Умова-тригер (Логічне if): (чітка бізнес-умова, напр.: Користувач обрав оплату готівкою замість картки)
- ✓ Повернення на крок Happy Path: (вказати номер кроку, куди повертаємо процес, напр. Крок №4)
- ✓ Покроковий алгоритм обробки:

Крок	Дія Актора (Заповнює BA)	Реакція та відповідь системи (Заповнює PM)
1.		
…		

ЗАВДАННЯ №4. Інженерний ШІ-краш-тест та специфікація руйнівних виключень (Exception Flows)

Мета: Навчитися проєктувати стійкі до аварій програми; використовувати ШІ як «суворого критика» для пошуку слабких місць у логіці софту на граничних випадках (Edge Cases); навчитися описувати сценарії критичних помилок (Exception Flows), коли процес повністю ламається через зовнішні чинники, та створювати алгоритми захисту даних.

Базовий інструментарій: Confluence (документ BRD, новий Розділ 4.4 «Сценарії виключень та обробка помилок прецеденту [ID]»), Notion (підсторінка 03_LLM_Logs), будь-яка LLM.

Покроковий алгоритм виконання:

Для розроблених Use Cases необхідно передбачити та розписати сценарії виключень, де мета сценарію не досягається (наприклад через відмову API, зник інтернет, Connection Timeout сервера тощо), але софт все-рівно має коректно обробити помилку, зберегти цілісність даних та повернути систему у безпечний стан. Для цього під кожен Use Case ініціалізується ШІ-краш-тест.

1. Запуск ШІ-краш-тесту (Стрес-сесія).

Команда повністю копіює вже готову документацію свого Use Case з Confluence (Реєстр каскадування з Розділу 4.1, Happy Path з Розділу 4.2 та Альтернативні потоки з Розділу 4.3) у чат із ШІ. Вони мають самостійно скласти промпт-виклик, який змусить ШІ діяти в ролі «прискіпливого та цинічного IT-архітектора». Завдання ШІ – знайти щонайменше 2–3 сценарії, коли система повністю ламається через технічні причини (наприклад: гроші з картки списалися, але в цей момент зник інтернет, або вимкнувся сервер ресторану і замовлення «зависло» в повітрі).

2. Проєктування сценаріїв збоїв (Exception Flows).

На основі знайдених ШІ прогалин команда розробляє алгоритм безпеки: що робить софт, якщо «все впало»? Мета – повернути систему у безпечний стан, щоб дані в базі не переплуталися, а користувач не втратив гроші. У Confluence (Розділ 4.4) додається таблиця специфікації збоїв за шаблоном.

- Студент А (BA): Проєктує ліву колонку таблиці. Він детально прописує текстові повідомлення для екрану користувача. Вони мають бути написані людською мовою (без системного коду помилок), містити чітку інструкцію та заспокоювати клієнта.
- Студент Б (PM): Проєктує праву колонку таблиці. Він описує логіку «під капотом»: які статуси змінюються в базі даних, скільки разів сервер повторює запит і як запускається команда скасування операції.

Специфікація критичного збою (Exception Flow Specification):

- ✓ Exception Flow ID: (наприклад, EF-01A)
- ✓ Назва збою: (наприклад: Відмова сервера ресторану після успішної оплати)
- ✓ Умова тригеру: (що саме зламалося, наприклад: Платіжна система повернула статус «Успішно», але сервер ресторану видав помилку зв'язку).

Приклад:

Крок	Дії для UI / Користувача (Заповнює BA)	Технічні дії сервера та БД (Заповнює PM)
1.	Клієнт бачить вікно: «Оплата успішна, але виникла помилка зв'язку. Зачекайте 30 секунд, ми перевіряємо дані».	Система змінює статус замовлення в базі даних на Помилка Синхронізації та автоматично пробує надіслати запит до ресторану ще 3 рази з інтервалом у 10 секунд.
2.	Якщо зв'язок не відновився, користувач бачить: «Ресторан недоступний. Гроші автоматично повернуто на вашу картку».	Якщо всі спроби невдалі, сервер скасовує замовлення, змінює його статус у базі на Скасовано Системою та надсилає автоматичний запит у банк на повернення грошей клієнту.

Примітка: Відмінність Exception Flow від Alternative Flow?

- Alternative Flow (Альтернативний шлях): Це коли користувач просто передумав або помилився в логіці, але мета все одно досягається. Наприклад: Клієнт хотів сплатити карткою, але на ній немає грошей. Система показала помилку, клієнт обрав «Оплата готівкою» і успішно отримав замовлення.
- Exception Flow (Сценарій збою): Це технічна катастрофа, коли мета Use Case вже НЕ може бути досягнута, процес повністю зламався. Наприклад: Клієнт сплатив замовлення, гроші знялися, але в цей момент повністю вимкнулося світло на сервері ресторану. Оформити замовлення фізично неможливо. Завдання Exception Flow – зробити так, щоб програма не «зависла», автоматично повернула клієнту гроші, записала помилку в базу даних і красиво вивела повідомлення на екран.

3. Фіксація логів у Notion.

На сторінці 03_LLM_Logs під toggle-спойлером із назвою «ЛР№5_Завд.4: Результати ШІ-аудиту бізнес-логіки та Edge Cases» команда розміщує текст промпту, який вони придумали для краш-тесту, та повний текст діалогу-суперечки з ШІ.

У лозі має бути чітко видно, які зауваження робота студенти прийняли та виправили в Confluence, а які – аргументовано відхилили.

КРИТЕРІЇ ОЦІНЮВАННЯ (Максимум – 10 балів)

Завдання	Критерії оцінювання та розподіл балів
Завдання №1. Системне каскадування (max: 2.0 б.)	2.0 б. (Відмінно): Студенти провели глибоку ревізію беклогу. У Confluence (Розділ 4.1) створено чітку таблицю каскадування Use Cases. Усі посилання на User Stories інтегровані через живі Jira Macro лінки. Обґрунтування меж прецедентів є логічним та системним. 1.0 б. (Задовільно): Групування виконано формально (1 Story = 1 Use Case, немає агрегації). Зв'язок із Jira налаштовано текстовими копіями ID замість живих макросів, або обґрунтування меж написане загальними абстрактними фразами. 0 б. (Незадовільно): Розділ 4.1 порожній, або реєстр прецедентів повністю скопійований з ШІ без прив'язки до реального беклогу команди з ЛР№4.
Завдання №2. Тран TRANSACTION-еталон та стані (max: 3.0 б.)	3.0 б. (Відмінно): В Розділі 4.2 заповнено Use Cases Specification для всіх прецедентів. Описано жорсткі інженерні контракти Preconditions та Postconditions. Покрокова таблиця Happy Path містить суворе розділення: опис бізнес-намірів та системні відповіді backend-сервера. 1.5 б. (Задовільно): Алгоритм Happy Path наявний, але реакції системи описані на рівні «система відкриває екран». Передумови та постумови сформульовані у «гуманітарному» стилі («клієнт задоволений», «їжа готується») замість оперування станами баз даних. 0 б. (Незадовільно): Блок специфікацій Happy Path та інженерних станів повністю відсутній або скопійований з чужих робіт.
Завдання №3. Алгоритмізація Alternative Flows (max: 2.5 б.)	2.5 б. (Відмінно): В Розділі 4.3 спроєктовано та заповнено Матрицю альтернативних потоків (не менше 2-х потоків Типу 1 та не менше 2-х потоків Типу 2 для кожного Use Cases). Чітко зафіксовано метадані: ID потоку, крок старту, умова-тригер (if) та точний крок повернення на Happy Path. Повністю відсутнє «інтерфейсне сміття». 1.25 б. (Задовільно): Зафіксовано менше 4 потоків для кожного Use Cases, або замість суворих законів серверної валідації студенти почали описувати помилки верстки фронтенду («користувач забув ввести пароль»). Порушено рольовий контур заповнення колонок таблиці. 0 б. (Незадовільно): Розділ Альтернативних сценаріїв у Confluence повністю порожній.
Завдання №4. ШІ-краш-тест та Exception Flows (max: 2.5 б.)	2.5 б. (Відмінно): Проведено стрес-сесію ШІ-аудиту. В Notion (підсторінка 03_LLM_Logs) зафіксовано лог-суперечку з ШІ, де наочно видно аналітичну роботу студентів. На основі критики кібераудитора в Confluence (Розділ 4.4) розписано таблицю руйнівних збоїв (Exception Flows). Специфіковано чіткі інженерні механізми захисту даних. Архітектурні рішення занесені до Decision Log. 1.25 б. (Задовільно): ШІ-аудит проведено, але його результати формально скопійовані без логу суперечки з ШІ (немає аналізу критики). Таблиця збоїв

	заповнена поверхнево, інженерні механізми обробки аварій сервера пропущені або замінені абстрактними порадами. 0 б. (Незадовільно): Логи краш-тесту відсутні, опрацювання сценаріїв виключень повністю проігноровано, Decision Log не оновлено.

ШТРАФНІ САНКЦІЇ (Критерії зниження балів під час захисту)

- Порушення тайм-менеджменту (–1 бал): Якщо під час Демо команда не вкладається в жорсткі регламентні 4 хвилини для кожного із блоків, викладач зупиняє презентацію на позначці таймера (навіть посеред слова) та знижує підсумкову оцінку.
- Порушення рольового контуру та синергії (–2 бали): Якщо один студент захищає та розказує абсолютно всі завдання, а його напарник не може відповісти на запитання щодо своєї зони відповідальності.
- Критерій «Інтерфейсного сміття» у Use Cases (–1.5 бала): Якщо в Розділі 4.3 (Alternative Flows) замість суворих доменних законів бізнес-валідації сервера будуть виявлені описи простих помилок фронтенду («користувач ввів 10 цифр замість 12», «не заповнено обов'язкове поле форми»). Такі помилки відсікаються верстkою і є дефектом проєктування системних вимог Enterprise-рівня.
- Критерій «Сліпого копіпасту з ШІ» (–2 бали): Якщо під час аналізу сторінки 03_LLM_Logs виявлено, що студенти просто скопіювали згенеровану ШІ таблицю без зміни інженерних термінів на простіші (відповідно до умов завдання), або якщо у логу відсутній процес «суперечки» з ШІ (команда сліпо погодилася з усіма галюцинаціями або помилками ШІ-архітектора без власної аналітики).